Rapport Projet IL S5

Projet IL 2025–2026

Plateforme collaborative de visionnage vidéo WithYou

Université d'Avignon
06 janvier 2026

Prestataires :
Takdjerad Meriem
Ghabi Malek
Taleb Wissame
Taleb Lamia
Laftimi Yanis

Clients :
KESSLER Rémy
BONNEFOY Ludovic

UFR Sciences, Technologies, Santé — CERI

Table des matières

Page de garde 1

1 Introduction 3

2 Description technique 4

2.1 Stack technologique 4
2.1.1 Front-end : React.js et Tailwind CSS 4
2.1.2 Back-end : PHP et Laravel 4
2.1.3 Base de données : PostgreSQL via Supabase 4
2.1.4 Communication temps réel : Laravel Reverb et Laravel Echo 4
2.1.5 API externe : YouTube IFrame API 5
2.1.6 Sécurité : JSON Web Token (JWT) 5
2.1.7 Outils collaboratifs 5

2.2 Architecture logicielle 5
2.2.1 Front-end (Client React) 5
2.2.2 Back-end (Serveur Laravel) 5
2.2.3 Base de données (PostgreSQL via Supabase) 5
2.2.4 Communication temps réel (Laravel Reverb et Echo) 6

3 Gestion de projet 7

3.1 Méthodologie 7

3.2 Cahier des charges et priorisation 7

3.3 Outils de gestion de projet 8

3.4 Répartition des rôles 8

4 Fonctionnalités développées 10

4.1 Page d'accueil 10

4.2 Salons publics 10

4.3 Création et gestion des salons 10

4.4 Interface de visionnage collaboratif 10

4.5 Synchronisation en temps réel 11

4.6 Chat en temps réel 11

4.7 Gestion de la playlist 11

4.8 Système de notation 11

4.9 Sondages 12

4.10 Historique des vidéos 12

5 Contributions individuelles **13**
5.1 Contribution de Meriem Takdjerad 13
5.2 Contribution de Malek Ghabi 14
5.3 Contribution de Yanis Laftimi 16
5.4 Contribution de Taleb Wissame 18
6 Problèmes rencontrés et solutions **21**
7 Tests et validation **23**
8 Conclusion **24**

1. Introduction

Dans le cadre de notre unité d'enseignement d'Ingénierie Logicielle, nous avons développé **WithYou**, une plateforme web collaborative permettant à plusieurs utilisateurs de regarder des vidéos YouTube ensemble de manière synchronisée et en temps réel.

Ce projet s'inspire de services existants tels que *Watch2Gether*, tout en y ajoutant des fonctionnalités interactives favorisant l'engagement et la prise de décision collective, notamment un chat instantané, un système de sondage permettant de choisir la prochaine vidéo à visionner, ainsi qu'un mécanisme de notation visant à évaluer la qualité des salons et de l'expérience proposée. La plateforme propose également des salons publics et privés, accessibles via un code d'invitation unique.

L'objectif principal de **WithYou** est d'offrir une expérience de visionnage partagé fluide, conviviale et participative, dans laquelle les utilisateurs peuvent créer des salons, inviter leurs amis, échanger en temps réel et gérer collectivement une playlist de vidéos.

Ce projet nous a permis d'explorer l'ensemble du cycle de développement logiciel, depuis l'analyse des besoins jusqu'à la mise en production, en passant par la conception de l'architecture, l'implémentation de fonctionnalités temps réel via les WebSockets, ainsi que la gestion collaborative du code source à l'aide de Git et GitHub.

Nous avons adopté une méthodologie de type cascade, adaptée au cadre académique, en suivant des phases structurées (analyse, conception, développement, tests), tout en nous appuyant sur des outils modernes de gestion de projet tels que *Jira* et *Notion* afin d'assurer une organisation rigoureuse et une progression régulière du travail.

2. Description technique

2.1 Stack technologique

Le projet **WithYou** a été développé en s'appuyant sur une stack technologique moderne, choisie afin de garantir la performance, la fiabilité et la maintenabilité de l'application.

2.1.1 Front-end : React.js et Tailwind CSS

L'interface utilisateur est développée avec *React.js*, une bibliothèque JavaScript permettant la création de composants réutilisables, dynamiques et modulaires. React facilite la gestion de l'état de l'application et la mise à jour efficace de l'interface en fonction des interactions des utilisateurs. Le design et la mise en forme sont assurés par *Tailwind CSS*, un framework utilitaire léger favorisant un rendu visuel cohérent, propre et responsive.

2.1.2 Back-end : PHP et Laravel

La partie serveur repose sur *PHP* avec le framework *Laravel*, qui permet de structurer une API RESTful sécurisée. Laravel fournit nativement des outils pour la gestion des routes, la validation des requêtes, l'authentification, la sécurité applicative et l'accès à la base de données.

2.1.3 Base de données : PostgreSQL via Supabase

Les données de l'application (utilisateurs, salons, messages, vidéos, playlists, votes, etc.) sont stockées dans une base *PostgreSQL* hébergée via *Supabase*. Ce choix garantit la robustesse, la cohérence et la persistance des données. Laravel interagit avec cette base à l'aide de son ORM *Eloquent*, facilitant la manipulation des entités applicatives.

2.1.4 Communication temps réel : Laravel Reverb et Laravel Echo

Les fonctionnalités temps réel (chat instantané, synchronisation de lecture vidéo, mise à jour des sondages) reposent sur l'utilisation de WebSockets. *Laravel Reverb* agit comme serveur de communication temps réel, tandis que *Laravel Echo* est utilisé côté client pour écouter les événements diffusés. Cette combinaison permet une communication bidirectionnelle fluide et instantanée entre les utilisateurs d'un même salon.

2.1.5 API externe : YouTube IFrame API

L'API *YouTube IFrame* est utilisée pour intégrer le lecteur vidéo directement dans l'interface de l'application. Elle permet de contrôler la lecture (lecture, pause, changement de vidéo) tout en garantissant une compatibilité et une performance optimales.

2.1.6 Sécurité : JSON Web Token (JWT)

L'authentification repose sur l'utilisation de *JSON Web Tokens*. Après connexion, chaque utilisateur reçoit un jeton signé qui est transmis lors des requêtes suivantes afin de garantir un accès sécurisé aux ressources protégées de l'application.

2.1.7 Outils collaboratifs

— **GitHub** : gestion du code source et versioning collaboratif ;
— **Jira** / **Notion** : planification des tâches, suivi d'avancement et documentation ;
— **Postman** : tests des routes de l'API ;
— **Figma** : conception des maquettes et des diagrammes (UML, interfaces).

2.2 Architecture logicielle

L'architecture de **WithYou** repose sur une séparation claire entre le client, le serveur et la couche de données, favorisant la modularité, la scalabilité et la maintenabilité du système.

2.2.1 Front-end (Client React)

Le front-end est une application React composée de composants réutilisables organisés par fonctionnalités. Il communique avec le back-end via une API RESTful pour les opérations classiques (authentification, création de salons, gestion des playlists, votes, etc.). Pour les interactions temps réel, *Laravel Echo* établit une connexion WebSocket avec le serveur afin de recevoir instantanément les événements liés aux salons (messages, synchronisation vidéo, changements d'état).

2.2.2 Back-end (Serveur Laravel)

Le serveur *Laravel* expose une API RESTful sécurisée par JWT. Il gère les utilisateurs, les salons, les playlists, l'historique et les sondages à l'aide de l'ORM *Eloquent*. Les événements temps réel (lecture, pause, changement de vidéo, messages) sont émis via *Laravel Reverb* et diffusés à l'ensemble des clients connectés à un même salon.

2.2.3 Base de données (PostgreSQL via Supabase)

PostgreSQL assure le stockage persistant des données de manière fiable et sécurisée. *Supabase* fournit l'infrastructure de la base ainsi que des mécanismes de sécurité et de gestion des accès, garantissant la protection des informations sensibles.

2.2.4 Communication temps réel (Laravel Reverb et Echo)

Laravel Reverb agit comme serveur de communication temps réel en gérant les connexions WebSocket et la diffusion des événements. *Laravel Echo* permet au client d'écouter ces événements et de mettre à jour l'interface utilisateur de manière instantanée.

Cette architecture garantit une séparation claire des responsabilités entre les différentes couches du système, facilite l'évolution indépendante de chaque composant et simplifie la maintenance globale de l'application.

3. Gestion de projet

3.1 Méthodologie

Pour développer la plateforme **WithYou**, nous avons suivi une démarche structurée de type cascade, en passant successivement par les phases d'analyse des besoins, de conception, de planification, de développement et de tests, tout en travaillant de manière collaborative.

La phase d'analyse a permis d'identifier les fonctionnalités attendues, les contraintes techniques et les objectifs du projet. Elle a servi de base pour définir un périmètre clair et éviter les dérives fonctionnelles.

La phase de conception a consisté à définir l'architecture générale du système (séparation front-end / back-end / base de données), ainsi que la structure des principales entités (utilisateurs, salons, messages, vidéos, playlists, votes).

La planification a ensuite permis de répartir les tâches entre les membres de l'équipe et d'ordonner leur réalisation en tenant compte des dépendances techniques entre les différentes parties du projet.

La phase de développement a été menée de manière progressive, chaque membre implémentant ses fonctionnalités, avec des intégrations régulières afin de maintenir une version fonctionnelle du projet.

Enfin, la phase de tests a permis de vérifier le bon fonctionnement de l'application, de corriger les erreurs détectées et d'améliorer la stabilité globale, en particulier pour les fonctionnalités temps réel et la gestion des sessions.

Cette organisation nous a permis de structurer efficacement notre travail et d'aboutir à une application cohérente et fonctionnelle.

3.2 Cahier des charges et priorisation

En début de projet, nous avons rédigé un cahier des charges recensant l'ensemble des fonctionnalités envisagées, puis nous les avons classées par ordre de priorité afin de garantir la livraison d'un produit fonctionnel dans les délais impartis.

Nous avons distingué trois niveaux de priorité :

Fonctionnalités essentielles (VIT — Vitales)

Ces fonctionnalités constituent le cœur du système et sont indispensables à son fonctionnement :

— Inscription et authentification sécurisée des utilisateurs
— Création et accès aux salons (publics ou privés)
— Invitation via lien ou code d'accès

— Lecture de vidéos YouTube via l'API officielle
— Synchronisation des actions principales (lecture, pause, changement de vidéo)
— Chat textuel en temps réel intégré au salon
— Gestion de base de la playlist (ajout, suppression, sélection)

Fonctionnalités importantes (IMP — Importantes)

Elles enrichissent l'expérience utilisateur sans être strictement nécessaires au fonctionnement minimal :
— Gestion des rôles et permissions dans un salon
— Système de sondage pour le choix des vidéos
— Notation du salon et de l'expérience utilisateur
— Profil utilisateur
— Améliorations de l'interface utilisateur

Fonctionnalités optionnelles (MIN — Minimales)

Elles constituent des améliorations complémentaires :
— Abonnement premium
— Multilingue
— Chat vocal
— Recommandation automatique de vidéos
— Mode sombre

Cette priorisation nous a permis de concentrer nos efforts sur les fonctionnalités essentielles et d'assurer la livraison d'un *MVP* (Minimum Viable Product) fonctionnel à la fin du semestre.

3.3 Outils de gestion de projet

Plusieurs outils ont été utilisés afin d'organiser le travail et de faciliter la collaboration au sein de l'équipe.
— **GitHub** : gestion du code source, versioning et intégration des contributions
— **Discord et WhatsApp** : communication, réunions, coordination et résolution des blocages
— **Jira et Notion** : planification, suivi d'avancement, diagrammes et documentation
— **Figma** : conception des maquettes et des interfaces utilisateur
— **Postman** : tests des routes de l'API

Ces outils ont permis de maintenir une bonne coordination, une répartition équilibrée du travail et une progression régulière du projet.

3.4 Répartition des rôles

La répartition des tâches au sein de l'équipe a été effectuée en fonction des compétences et des préférences de chacun, tout en veillant à ce que chaque membre participe à différents aspects du projet afin de conserver une vision globale du système.

Équipe Back-end (Taleb Wissame & Yanis Laftimi)

- Développement de l'API RESTful avec Laravel
- Mise en place de l'authentification JWT
- Configuration de la base de données PostgreSQL via Supabase
- Création des modèles Eloquent
- Développement des contrôleurs et des routes API
- Implémentation de la logique métier

Équipe Front-end (Meriem Takdjerad & Malek Ghabi)

- Développement de l'interface avec React et TypeScript
- Intégration de Tailwind CSS
- Création des pages principales et des composants réutilisables
- Gestion de l'état global avec React Context et Hooks
- Harmonisation visuelle de l'application

Travail collaboratif

- Intégration front-end / back-end
- Configuration de Laravel Reverb et Echo
- Mise en place du système de broadcasting
- Tests multi-utilisateurs
- Débogage collaboratif (WebSockets, CORS, JWT)

Des réunions hebdomadaires ont permis de synchroniser l'avancement, résoudre les blocages et ajuster la répartition des tâches en fonction des besoins du projet.

4. Fonctionnalités développées

4.1 Page d'accueil

La page d'accueil constitue le point d'entrée de la plateforme **WithYou**. Elle permet aux utilisateurs de comprendre rapidement le concept du service et d'accéder aux principales fonctionnalités.

Elle propose :

— une présentation synthétique du concept de visionnage collaboratif ;
— des accès directs vers la création et la recherche de salons ;
— l'authentification des utilisateurs ;
— une interface responsive compatible mobile, tablette et ordinateur.

Technologies : React, TypeScript, Tailwind CSS, React Router.

4.2 Salons publics

Une page dédiée permet de consulter et rechercher les salons publics disponibles.

Fonctionnalités :

— affichage de la liste des salons en cours ;
— recherche par nom de salon ;
— affichage du nombre de participants et du statut (en direct) ;
— accès direct à un salon public.

4.3 Création et gestion des salons

Les utilisateurs peuvent créer un salon de visionnage personnalisé.

Fonctionnalités :

— création de salon avec nom, description et vidéo initiale optionnelle ;
— choix entre salon public ou privé (avec mot de passe) ;
— génération automatique d'un code d'invitation ;
— limitation du nombre maximum de participants ;
— gestion des rôles (administrateur / participant) ;
— gestion des accès et permissions.

Technologies : Laravel, PostgreSQL (Supabase), React, YouTube IFrame API.

4.4 Interface de visionnage collaboratif

Cette interface constitue le cœur de la plateforme.

Elle comprend :
— un lecteur YouTube synchronisé ;
— une playlist interactive ;
— un panneau latéral regroupant le chat, les participants, les sondages, la notation et l'historique des vidéos ;
— des contrôles spécifiques pour l'administrateur ;
— un design moderne en thème sombre et responsive.

4.5 Synchronisation en temps réel

Les actions vidéo (lecture, pause, changement de vidéo) sont synchronisées entre tous les participants via des événements temps réel.

L'infrastructure est opérationnelle et en cours de stabilisation afin d'assurer une propagation fiable des événements.

Technologies : Laravel Reverb, Laravel Echo, WebSockets.

4.6 Chat en temps réel

Les utilisateurs peuvent échanger des messages pendant le visionnage.
Fonctionnalités :
— messagerie instantanée ;
— réactions par emoji ;
— affichage du pseudonyme, de l'heure et distinction visuelle des messages ;
— défilement automatique des messages.

4.7 Gestion de la playlist

Les participants peuvent gérer collectivement une liste de vidéos.
Fonctionnalités :
— ajout de vidéos via URL YouTube ;
— extraction automatique des informations ;
— affichage sous forme de grille ;
— suppression réservée à l'administrateur ;
— persistance en base de données.

4.8 Système de notation

Les utilisateurs peuvent attribuer une note aux vidéos visionnées.
Fonctionnalités :
— notation sur 5 étoiles ;
— calcul automatique de la moyenne ;
— modification possible des notes ;
— affichage dédié des évaluations.

4.9 Sondages

L'administrateur peut créer des sondages interactifs.
Fonctionnalités :
— création de sondages avec options ;
— vote unique par utilisateur ;
— affichage des résultats en temps réel ;
— conservation de l'historique des sondages.

4.10 Historique des vidéos

Un historique des vidéos visionnées est conservé pour chaque salon.
Fonctionnalités :
— enregistrement automatique des vidéos lues ;
— affichage chronologique ;
— possibilité de relancer une vidéo depuis l'historique.

5. Contributions individuelles

5.1 Contribution de Meriem Takdjerad

Rôle et responsabilités

Meriem Takdjerad a initialement participé au projet en tant que développeuse front-end, en charge de la conception et de l'implémentation des pages principales de l'application. Cette première phase visait à construire une interface complète et cohérente, avec des parcours utilisateurs fonctionnels, en s'appuyant sur des données simulées afin de valider la structure globale de l'application.

Dans une seconde phase, Meriem a élargi son rôle vers une contribution front-end et back-end ciblée, principalement autour de la sécurité, de l'authentification et de la fonctionnalité sondages. Elle a intégré les routes d'authentification existantes, puis développé des fonctionnalités back-end spécifiques (validation d'email, réinitialisation de mot de passe, sondages).

Ses responsabilités principales ont été :

- développement des pages front-end et des layouts globaux ;
- intégration et structuration des composants UI réutilisables ;
- intégration des routes d'authentification (login / register) côté front-end ;
- implémentation de la vérification d'email obligatoire avant connexion ;
- développement du workflow complet de réinitialisation de mot de passe ;
- ajout de logique back-end dédiée (contrôleurs et routes API) pour la gestion des emails ;
- configuration et débogage de l'envoi d'emails (SMTP) ;
- développement et connexion de la fonctionnalité sondages ;
- correction et connexion des fonctionnalités d'accès aux salons (rejoindre un salon existant).

Réalisations concrètes

1) Développement front-end Meriem a conçu et intégré plusieurs pages côté client, notamment :

- Pages d'authentification : SignUpPage, SignInPage, ForgotPasswordPage, EmailSentPage, AccountConfirmedPage, ResetPasswordPage ;
- Pages informatives et légales : Conditions d'utilisation, Politique de confidentialité, Règles du salon, FAQ, Contact ;
- Layouts globaux : Header, Footer ;
- Pages et composants de paramètres des salons (sondages, permissions, membres, informations).

2) Intégration UI Meriem a mis en place une base complète de composants UI (Form, Button, Input, Dialog, Tabs, NavigationMenu, DropdownMenu, Sheet, etc.), permettant une interface homogène, réutilisable et maintenable dans l'ensemble de l'application.

3) Authentification et sécurité Meriem a intégré les routes login et register existantes, puis ajouté :
- la vérification de l'email après inscription ;
- la protection de la connexion tant que l'email n'est pas validé ;
- les routes API associées.

4) Réinitialisation de mot de passe (Front + Back) Elle a développé le workflow complet :
- demande de réinitialisation via le front-end ;
- envoi d'un email contenant un lien sécurisé ;
- validation du lien et modification effective du mot de passe côté back-end.

5) Fonctionnalité sondages Meriem a développé la fonctionnalité sondages de manière full stack :
- création de la table sondage en base ;
- création du contrôleur associé ;
- définition des routes API ;
- connexion au front-end.

6) Accès aux salons Elle a corrigé et connecté la page JoinRoomPage pour permettre à un utilisateur de rejoindre un salon existant via le back-end.

Problèmes rencontrés

- coordination entre front-end, back-end et base de données ;
- configuration et débogage du service SMTP ;
- adaptation du front-end lors du passage de données simulées à l'API réelle ;
- intégration simultanée front/back des sondages.

Apports spécifiques

Meriem a apporté une contribution significative au projet, permettant :
- la mise en place d'un front-end complet et structuré ;
- un parcours utilisateur sécurisé (validation email, reset password) ;
- l'intégration d'une fonctionnalité métier persistante (sondages) ;
- une évolution progressive vers des tâches full stack ciblées.

5.2 Contribution de Malek Ghabi

Rôle et responsabilités

Malek Ghabi a participé au projet en tant que développeuse front-end, avec la responsabilité principale de la conception de l'expérience utilisateur (UX/UI) et de l'implémentation technique des interfaces de l'application. Son rôle a couvert l'ensemble du cycle

de développement front-end, depuis la conception des maquettes jusqu'à l'intégration des fonctionnalités avec le back-end.

Elle a assuré le passage des maquettes Figma au code applicatif en React, en veillant au respect des principes de modularité, de réutilisabilité des composants et de maintenabilité du code.

En parallèle, Malek a contribué à l'organisation et au suivi du projet à l'aide d'outils de gestion tels que Jira, Notion et Excel, notamment pour la planification des tâches, le suivi de l'avancement et la préparation de la soutenance.

Réalisations concrètes

1) Conception et développement front-end Malek a conçu et développé les interfaces principales de l'application, en particulier celles liées aux salons (rooms), qui constituent le cœur fonctionnel de la plateforme. Elle a notamment implémenté :

- la page d'accueil et la navigation globale ;
- les interfaces liées aux salons :
  - liste des salons publics ;
  - interface de création de salon ;
  - page permettant de rejoindre un salon par code ;
  - page principale d'un salon (RoomPage) et pages de paramètres associées ;
- les interfaces de gestion des vidéos :
  - ajout de vidéos à la playlist ;
  - suppression de vidéos par l'administrateur ;
  - affichage de la vidéo en cours et des vidéos à venir ;
  - interface administrateur associée ;
  - page de profil utilisateur.

Ces développements ont permis de structurer l'ensemble du parcours utilisateur, depuis l'entrée sur la plateforme jusqu'à l'utilisation collaborative complète à l'intérieur d'un salon.

2) Pages et composants avancés des salons Malek a implémenté l'ensemble des composants front-end liés aux salons, notamment :

- intégration du lecteur vidéo YouTube ;
- gestion de la playlist ;
- affichage et gestion des participants ;
- gestion des permissions et des règles du salon ;
- affichage des informations du salon ;
- interface de notation des vidéos ;
- interface de vote des sondages ;
- historique des vidéos ;
- page d'invitation avec lien et code de salon.

3) Intégration UI et design system Malek a mis en place et exploité une base cohérente de composants UI (Form, Button, Input, Dialog, Tabs, NavigationMenu, DropdownMenu, Sheet, etc.), garantissant une interface homogène, cohérente et responsive.

Elle a également contribué à l'harmonisation du design system (typographie, couleurs, espacements, comportements interactifs).

4) Architecture front-end Malek a participé à :
- l'organisation de l'architecture front-end ;
- la mise en place du routage applicatif ;
- la documentation de la structure front-end.

5) Intégration front-end / back-end Malek a contribué à la connexion des interfaces front-end aux services back-end, notamment pour :
- la récupération et l'actualisation de la liste des salons publics ;
- la gestion des permissions des participants ;
- l'intégration de la notation des vidéos ;
- la participation à l'intégration des sondages (vote) ;
- la connexion de la page profil utilisateur ;
- la gestion des réponses serveur (succès, erreurs, messages).

6) Tests et amélioration continue Malek a participé aux tests manuels, aux tests multi-utilisateurs, à la correction d'anomalies front-end, ainsi qu'à la préparation de la démonstration pour la soutenance finale.

Problèmes rencontrés

- coordination entre les différentes couches de l'application ;
- gestion de la complexité croissante des composants ;
- ajustement progressif des interactions dynamiques.

Ces difficultés ont été résolues par un travail itératif de conception, de test, de correction et de concertation avec les autres membres de l'équipe.

Apports spécifiques

La contribution de Malek Ghabi a permis :
- la mise en place d'un front-end complet, structuré et ergonomique ;
- une expérience utilisateur fluide et cohérente, centrée sur les salons ;
- une intégration efficace avec le back-end ;
- une meilleure organisation du travail d'équipe ;
- une base front-end solide pour les évolutions futures.

5.3 Contribution de Yanis Laftimi

Rôle et responsabilités

Yanis Laftimi a participé au projet en tant que développeur principalement back-end, avec une forte implication sur la structure technique globale de l'application et sur les fonctionnalités temps réel. Son rôle a consisté à poser et stabiliser les fondations techniques du projet, tout en développant des fonctionnalités centrales liées aux utilisateurs, aux salons et aux interactions en temps réel.

Il est intervenu dès les premières phases du projet afin de structurer l'architecture back-end, de définir les règles métier essentielles et d'assurer une communication fiable entre le front-end et le serveur. Une attention particulière a été portée à la cohérence des

états utilisateurs, notamment dans le cadre de la gestion des salons et des fonctionnalités temps réel (chat et vidéo).

Ses responsabilités principales ont été :

— mise en place de la structure technique back-end du projet ;
— développement et intégration des routes d'authentification ;
— implémentation de la logique métier liée aux salons ;
— gestion des connexions et déconnexions des utilisateurs aux salons ;
— mise en place du système de chat en temps réel ;
— participation à l'implémentation et à la synchronisation du temps réel vidéo ;
— création de pages de test front-end pour la validation des flux API ;
— tests, débogage et correction des incohérences techniques.

Réalisations concrètes

1) Mise en place de l'architecture back-end Yanis a participé à la définition et à la mise en place de l'architecture back-end de l'application, en veillant à une séparation claire entre la logique métier, les contrôleurs et les routes API.

Ce travail a permis :

— une meilleure lisibilité du code ;
— une maintenance facilitée ;
— une base évolutive pour l'ajout de nouvelles fonctionnalités.

L'architecture mise en place sert de socle à l'ensemble des fonctionnalités développées par l'équipe.

2) Gestion des utilisateurs et authentification Yanis a travaillé sur l'intégration et le bon fonctionnement du système d'authentification côté back-end, incluant :

— la gestion des routes d'inscription et de connexion ;
— la récupération des informations de l'utilisateur authentifié ;
— la sécurisation de l'accès aux fonctionnalités sensibles.

Cette partie est essentielle pour garantir que seuls les utilisateurs authentifiés puissent accéder aux salons et aux fonctionnalités temps réel.

3) Gestion des salons et logique métier La gestion des salons constitue un élément central du projet. Yanis a développé la logique back-end associée aux salons, incluant :

— la création et la gestion des entités salon ;
— la connexion et la déconnexion des utilisateurs ;
— la gestion des états utilisateurs liés aux salons.

Une règle métier clé a été implémentée : un utilisateur ne peut être connecté qu'à un seul salon à la fois, afin d'éviter les incohérences dans la gestion du chat et de la vidéo en temps réel.

4) Chat en temps réel Yanis a mis en place le système de chat en temps réel permettant aux utilisateurs d'échanger instantanément des messages au sein d'un salon. Cette fonctionnalité repose sur :

— l'envoi d'événements temps réel ;
— la diffusion des messages à l'ensemble des utilisateurs connectés au même salon ;
— l'association stricte des messages à un salon donné.

5) Temps réel vidéo Yanis a contribué à la mise en place de la synchronisation vidéo en temps réel, visant à garantir une expérience partagée cohérente entre les utilisateurs d'un même salon. Le travail réalisé a porté sur :

— la synchronisation des actions vidéo (lecture, pause, changements d'état) ;
— la gestion cohérente des événements temps réel ;
— l'interaction entre la vidéo et la logique des salons.

Cette partie représente l'un des aspects les plus techniques du projet.

6) Pages de test et validation des flux Afin de valider le bon fonctionnement des fonctionnalités, Yanis a développé des pages de test front-end permettant de :

— tester les routes d'authentification ;
— vérifier la gestion des salons ;
— valider le fonctionnement du chat et du temps réel ;
— contrôler la communication entre le front-end et le back-end.

Ces pages ont joué un rôle important dans le débogage et la stabilisation du projet.

Problèmes rencontrés

— gestion des états utilisateurs dans un contexte temps réel ;
— synchronisation des événements entre plusieurs utilisateurs connectés simultanément ;
— coordination entre logique métier, API et temps réel ;
— débogage des interactions front-end / back-end.

Ces difficultés ont été résolues par une approche itérative combinant tests, corrections progressives et ajustements de la logique serveur.

Apports spécifiques

La contribution de Yanis Laftimi a permis :

— la mise en place d'une base technique back-end solide et cohérente ;
— une gestion fiable des utilisateurs et des salons ;
— l'implémentation des fonctionnalités temps réel clés (chat et vidéo) ;
— une meilleure stabilité globale de l'application ;
— la création d'un socle technique facilitant les évolutions futures du projet.

5.4 Contribution de Taleb Wissame

Rôle et responsabilités

Taleb Wissame a principalement travaillé sur le développement back-end de l'application, en se concentrant sur l'architecture de la base de données, l'implémentation des fonctionnalités critiques et la communication temps réel. Il a également assuré l'intégration entre le back-end et le front-end pour plusieurs fonctionnalités majeures.

Réalisations concrètes

Architecture de la base de données Taleb a conçu et mis en place la structure complète de la base de données, incluant la création et la modification des tables (salons,

playlists, vidéos, historique, sondages, notations), la définition des relations entre les modèles Eloquent et la gestion des migrations. Il a également créé les modèles Laravel correspondants (`Salon`, `Playlist`, `Video`, `HistoriqueVideo`, `SondageSalon`, `PlaylistVideo`) avec leurs relations et méthodes associées.

Système de salons publics et privés Il a développé le système complet de gestion des salons avec deux types distincts : salons publics (accessibles via une liste) et salons privés (protégés par mot de passe). Il a implémenté la génération automatique de codes d'invitation uniques (format `XXXX-XXXX`), créé les routes API de création et d'accès aux salons (`POST /salons`, `GET /salons/by-code/{code}`), et développé la logique de vérification du mot de passe pour les salons privés.

Synchronisation vidéo en temps réel (front de test) Taleb a travaillé sur la synchronisation vidéo en temps réel en implémentant le système de diffusion des événements vidéo (lecture, pause, seek, changement de vidéo) via Laravel Reverb. Il a créé l'événement `VideoUpdated`, développé le contrôleur `VideoSyncController`, et mis en place le système de sauvegarde de l'état vidéo en base de données (position actuelle, statut lecture/pause).

Fonctionnalité historique des vidéos Il a développé le système complet d'historique permettant d'enregistrer automatiquement chaque vidéo lue dans un salon. Cela inclut la création de la table `historiquevideos` et du modèle associé, le développement du contrôleur `HistoriqueController` avec les routes d'enregistrement et de récupération, ainsi que l'enregistrement automatique des métadonnées (ID YouTube, titre, utilisateur, date et heure).

Système de playlist Il a implémenté la gestion complète des playlists avec la création de la table de liaison `playlist_video`, le développement des routes API d'ajout, de suppression et de récupération des vidéos, ainsi que la gestion du chargement automatique de la vidéo initiale lors de la création d'un salon.

Système de sondages Taleb a créé le système de sondages interactifs incluant la table `sondage_salon` et le modèle `SondageSalon`, les routes API de création, de vote et de récupération des résultats, ainsi que le calcul des statistiques (nombre de votes, classement des options).

Système de notation des vidéos Il a développé le système de notation permettant aux utilisateurs d'évaluer les vidéos sur cinq étoiles, avec l'enregistrement des notes en base de données, le calcul automatique de la note moyenne par vidéo et la possibilité de modifier sa note.

Configuration et tests Taleb a configuré le système CORS pour autoriser les communications entre le front-end et le back-end, configuré Laravel Reverb et les canaux WebSocket, créé des pages HTML de test pour valider l'authentification, les salons, la synchronisation vidéo et les WebSockets, et effectué des tests approfondis de l'ensemble des fonctionnalités développées.

Intégration back-end / front-end Il a assuré l'intégration de plusieurs fonctionnalités entre le back-end et le front-end, notamment la gestion des salons (création, accès par code), la récupération de l'historique des vidéos et le début de l'intégration de la playlist (en cours de finalisation).

Technologies utilisées

Laravel (API RESTful, Eloquent ORM, migrations), PostgreSQL / Supabase (base de données), Laravel Reverb (WebSocket), JWT (authentification), PHP (logique back-end), HTML / JavaScript (pages de test).

Problèmes rencontrés

La configuration initiale de Laravel Reverb et des WebSockets a nécessité plusieurs ajustements afin d'assurer une communication stable. La gestion de la synchronisation vidéo multi-utilisateurs a demandé de nombreux tests pour garantir la cohérence des états. La configuration CORS a également posé des difficultés initiales pour permettre les échanges entre ports différents (5173 côté front-end, 8000 côté back-end).

Apports spécifiques

Taleb Wissame a posé les fondations solides du back-end en créant une architecture de base de données cohérente et évolutive. Son travail sur les systèmes de salons, de synchronisation vidéo, d'historique, de playlists, de sondages et de notations a permis de couvrir l'ensemble des fonctionnalités métier critiques du projet. Ses pages de test ont grandement facilité le débogage et la validation des fonctionnalités avant l'intégration finale avec le front-end React.

6. Problèmes rencontrés et solutions

Au cours du projet, l'équipe a rencontré plusieurs difficultés, tant sur le plan technique qu'organisationnel. Les principaux problèmes identifiés ainsi que les solutions apportées sont présentés ci-dessous.

Configuration de Laravel Reverb et des WebSockets Le serveur WebSocket ne se connectait pas correctement au front-end et les événements diffusés n'étaient pas reçus. **Solution :** configuration des variables d'environnement (ports, clés), intégration de Laravel Echo avec le protocole Pusher et mise en place de tests progressifs.

Synchronisation vidéo Les événements de lecture (lecture, pause, changement de vidéo) étaient parfois perdus ou reçus avec un décalage. **Solution :** implémentation d'un événement dédié `VideoUpdated`, ajout de logs pour tracer les échanges et création de pages HTML de test pour valider les WebSockets.

Problèmes CORS Les requêtes entre le front-end (port 5173) et le back-end (port 8000) étaient bloquées par la politique de sécurité du navigateur. **Solution :** configuration du fichier `cors.php` afin d'autoriser les origines locales et activation du middleware CORS.

Envoi d'e-mails de vérification Les e-mails de confirmation n'étaient pas envoyés correctement. **Solution :** configuration des paramètres SMTP dans le fichier `.env` et utilisation d'un service de test fiable (Mailtrap).

Types de données PostgreSQL Des erreurs SQL sont apparues concernant les UUID et les horodatages automatiques. **Solution :** désactivation des timestamps sur certains modèles (`public $timestamps = false`) et utilisation correcte de `Str::uuid()`.

Intégration front-end / back-end Les formats de données entre les mocks front-end et l'API réelle n'étaient pas cohérents. **Solution :** définition claire des contrats d'API, typage TypeScript des interfaces et synchronisation régulière entre les équipes.

Conflits Git et gestion des versions Des conflits sont apparus lors des fusions et certains commits ont été perdus. **Solution :** adoption de bonnes pratiques Git (branches de fonctionnalités, commits fréquents, `git pull -rebase`) et sessions de sensibilisation au sein de l'équipe.

Retard initial Une phase d'apprentissage des technologies (Laravel, React, WebSockets) a ralenti le démarrage. **Solution :** réorganisation du planning en sprints courts et priorisation du MVP (Minimum Viable Product).

Mesures préventives mises en place

- Réunions hebdomadaires d'avancement
- Documentation des solutions (README, Notion)
- Backlog partagé (Jira / Notion)
- Pages de test HTML pour valider les fonctionnalités isolément
- Revue de code avant chaque fusion

7. Tests et validation

Conformément à la méthodologie en cascade adoptée, les phases de test ont été réalisées principalement en fin de développement, après l'implémentation des fonctionnalités principales. Les tests ont été effectués essentiellement de manière manuelle.

Tests fonctionnels manuels Chaque fonctionnalité clé (authentification, création de salons, playlist, chat, sondages, notation, historique) a été testée pas à pas afin de vérifier la cohérence des parcours utilisateurs, la gestion des erreurs (champs manquants, identifiants invalides, codes erronés) et la persistance des données dans PostgreSQL / Supabase.

Tests d'intégration API Les routes back-end ont été testées à l'aide de Postman et de pages HTML dédiées afin de vérifier les réponses JSON, l'authentification JWT et la cohérence des échanges entre le front-end et le back-end.

Tests de communication temps réel La synchronisation vidéo et le chat ont été testés en ouvrant plusieurs sessions simultanées afin de valider la réception des événements WebSocket, y compris dans des situations limites (déconnexion, reconnexion, utilisateurs multiples).

Limites et perspectives L'absence de tests automatisés a rendu le débogage plus long en fin de projet. La synchronisation vidéo reste partiellement stabilisée et nécessite encore des ajustements. Une approche plus itérative et l'ajout de tests unitaires et d'intégration automatisés permettraient d'améliorer la fiabilité et la maintenabilité de l'application.

8. Conclusion

Le projet **WithYou** constitue une plateforme collaborative fonctionnelle et cohérente. Les fonctionnalités principales sont opérationnelles : authentification sécurisée avec vérification par e-mail, création de salons publics et privés avec codes d'invitation, interface de visionnage collaboratif, chat en temps réel, gestion de playlists, sondages interactifs, système de notation et historique des vidéos.

Bien que l'infrastructure de synchronisation temps réel soit en place (Laravel Reverb et Laravel Echo), des ajustements sont encore nécessaires pour garantir une propagation parfaitement fiable des événements. De légères améliorations restent également à apporter à la gestion de la playlist.

Ce projet nous a permis de mettre en pratique les principes de l'ingénierie logicielle : analyse des besoins, conception d'architecture, développement full-stack, gestion collaborative du code et résolution de problématiques techniques complexes.

WithYou constitue aujourd'hui une base solide, évolutive et exploitable, qui pourra être enrichie par de nouvelles fonctionnalités en fonction des retours utilisateurs. Ce projet reflète notre progression collective et représente une réalisation dont nous sommes fiers.